Week 06: Subqueries and CTEs

Learning outcomes: CLO3, CLO6

Lesson

- **Scalar subquery:** one value, e.g. `WHERE area_km2 > (SELECT AVG(area_km2) FROM province)`.
- `IN (subquery)`: value is in the list. `EXISTS (subquery)`: at least one match; never repeats outer rows.
- **Correlated subquery:** uses the outer row's values.
- **Derived table:** named subquery in FROM. **CTE:** `WITH name AS (...)`, same idea, easier to read.
- Two ways to write the same question? Compare their outputs.

Worked example

Run `example.sql` (how (`setup/README.md`)), then answer in `work.sql`.

Tables for this week

Read `cambodia`, write `student_work`. All columns are `NOT NULL`. Samples are the first two rows by code.

`cambodia.province`

Column	Type	Key	Meaning
`province_pcode`	`CHAR(4)`	PK	Province code
`province_name`	`VARCHAR(100)`		Province name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place

Sample rows:

province_pcode	province_name	area_km2
KH01	Banteay Meanchey	6145.9
KH02	Battambang	11869.0

`cambodia.district`

Column	Type	Key	Meaning
`district_pcode`	`CHAR(6)`	PK	District code
`district_name`	`VARCHAR(100)`		District name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place
`province_pcode`	`CHAR(4)`	FK	Province code

Sample rows:

district_pcode	district_name	area_km2	province_pcode
KH0102	Mongkol Borei	807.2	KH01
KH0103	Phnum Srok	761.7	KH01

Join keys: `district.province_pcode = province.province_pcode`.

More: data guide (`data/README.md`).

Lab tasks

1. Find provinces above the average province area (scalar subquery).
2. Find provinces with at least one district, first with IN, then with EXISTS. Compare.
3. Find districts larger than their own province's average district (correlated).
4. With a CTE, count districts per province and keep counts above the average count.
5. Rewrite task 4 with a derived table and compare.

Homework

1. Answer one question with JOIN and with EXISTS. When does JOIN duplicate rows?
2. Review a classmate's schema: two suggestions.

Capstone sprint

C6: Add two multi-step queries. Record schema feedback and what you changed.

Submit

Answers in `work.sql`, reflection in `reflection.md`, output in `results/`. Commit and push. See SUBMISSIONS.md.